Identifying Large Language Models from Their Responses

CMPSC 448 Midterm Project | Individual submission

Author: Chengwei Fan
Report date: October 10, 2026

Abstract

This project studies whether a small text classifier can identify which of five large language models generated an answer. A balanced subset of LLMRouterBench contains 57,565 responses to 11,513 shared prompts. Prompt groups, including detected near duplicates, remain within one train, validation, or test split. A convolutional neural network (CNN) and a bidirectional LSTM (BiLSTM) are trained from scratch to predict the generator. On the 8,635-response test set, output-only classification reaches mean accuracy of 86.71% for the CNN and 85.67% for the BiLSTM across three seeds. Input-only accuracy is exactly 20%, as required by the paired design; adding the prompt provides no consistent CNN improvement and lowers BiLSTM macro-F1. Holding out a target domain during training reduces CNN macro-F1 from 0.8769 to 0.5187 for coding and from 0.7580 to 0.3599 for factual question answering. Formatting removal lowers performance, while masking selected model and provider names does not yield a detectable reduction. Attribution remains strong on a restricted, fixed-length subset. These results support learnable source cues within the sampled distribution, with substantial dependence on task coverage and response structure.

1 Problem and research questions

The objective is source attribution: given a response, predict its generator label. It is not a comparison of which generator answers questions most correctly. The five source LLMs are not retrained; the trainable systems are two approximately four-million-parameter classifiers. The study addresses the four questions in the course project specification [1].

RQ1 Can a CNN and an RNN identify the source model from output text alone?

RQ2 How does classification change when the classifier receives only the prompt, only the response, or both?

RQ3 Do the learned source cues transfer to a task domain excluded from training?

RQ4 Which observable linguistic and structural properties carry source information, and how do formatting, keyword masking, and length controls affect performance?

The task is closed-set classification among five specified model versions. It does not evaluate unknown generators, human authorship, external editing attacks, or later releases of the same model families.

2 Data source and curation

The data are existing model responses released by the authors of LLMRouterBench [2, 3], a benchmark originally developed for model routing. This project repurposes its response records for source classification. All formal experiments use the frozen response release specified in Section 11. No new API responses were generated for this study. The prepared record contains the source label, visible input prompt, original raw output, identifiers, benchmark, domain, and split; only the chosen text fields are fed to the classifier.

Table 1. Generator labels and exact source identifiers. Each source contributes 11,513 responses.

Label	Exact model identifier
GPT	gpt-5-chat
Claude	claude-sonnet-4
Gemini	gemini-2.5-flash
Qwen	qwen3-235b-a22b-2507
DeepSeek	deepseek-v3-0324

The starting selection contains 58,340 responses: 11,668 prompts answered by all five sources, from eight benchmarks. Overlapping subset exports are not added again. Tool-use trajectories and grid tasks are excluded. The original raw_output field is used because a processed prediction field can contain only an extracted answer. Correct answers, incorrect answers, short answers, and genuine refusals are retained; answer correctness is not a selection criterion.

Cleaning removes entire five-answer prompt groups to preserve balance: 152 groups contain collection-failure strings from Qwen, and one group has nonidentical visible prompt text across sources. Normalized exact matching then removes two duplicate prompts, leaving 11,513 prompts. Normalization uses Unicode NFKC, case folding, and whitespace collapsing for matching; delivered text is not rewritten. The exclusion of failed Qwen generations conditions the resulting sample on successful collection and may affect representativeness.

Table 2. Domain composition of the final corpus. Domain balance is not imposed.

Domain	Prompts	Responses
Knowledge and reasoning	5,223	26,115
Factual QA	4,312	21,560
Coding	1,303	6,515
Mathematics	425	2,125
Creative writing	250	1,250
Total	11,513	57,565

The eight benchmarks are AIME (59 prompts), LiveMathBench (121), LiveCodeBench (1,055), GPQA (198), HLE (2,040), MMLU-Pro (2,985), SimpleQA (4,312), and ArenaHard (743). Domain labels come from benchmark assignments and the source authors' ArenaHard subsets, not inferred answer style. They are broad task categories rather than mutually exclusive semantic topics.

3 Split design and text representation

Splitting individual response rows would allow answers to the same question into both training and testing. Instead, all five answers to a prompt are kept together. Additional connections join original queries of at least 80 characters with character 3-to-5-gram TF-IDF cosine similarity of at least 0.95, and identical normalized responses of at least 50 whitespace words. The resulting 11,461 connected groups are split approximately 70/15/15 with benchmark stratification and data seed 20261005. Lexical grouping reduces known overlap; it cannot certify the absence of every semantic paraphrase.

Table 3. Fixed splits and class balance.

Split	Responses	Prompts	Groups	Per source
Train	40,295	8,059	8,007	8,059
Validation	8,635	1,727	1,727	1,727
Test	8,635	1,727	1,727	1,727

Audit checks confirm complete five-source prompt groups, identical visible prompts within each group, fixed label mapping, and no shared record, prompt, connected group, or normalized exact input across splits. Recovered copies of all three prepared CSVs match the SHA256 hashes recorded during the formal run. This makes the later integrity review independent of the original saved audit alone.

The tokenizer lowercases Unicode word and digit sequences, retains punctuation and underscores, and represents newlines explicitly. It is a project tokenizer, not an API provider’s tokenizer. The vocabulary contains at most 30,000 tokens including padding, unknown, separator, newline, and identity markers. It is fitted only on the permitted training data. Within a training population, vocabulary construction uses training prompts and response prefixes and is shared across input modes; validation and test text never contribute vocabulary counts.

Table 4. Input conditions. The response budget is held constant when the prompt is added.

Condition	Observed tokens
Input only	First 256 prompt tokens
Output only	First 512 response tokens
Input plus output	First 256 prompt tokens + separator + first 512 response tokens

Labels, model identifiers, task/domain labels, filenames, and split/group IDs are not concatenated into the classifier text. Original responses remain available for auditing, but neural training sees only the specified prefixes. In training, 34.22% of responses exceed 512 tokens (24.16% to 45.02% across sources), so truncation is a material scope restriction. The maximum combined sequence is 769 tokens. Padding is excluded from CNN pooling windows and BiLSTM recurrent states so batch padding does not become a source cue.

3 1 Why the input only control must score 20 percent

Each prompt appears with each of the five source labels. A deterministic classifier receiving only that identical prompt must predict the same class for all five rows. Exactly one prediction is correct, regardless of which class it chooses. Thus 20% accuracy is a structural negative control. Macro-F1 can differ from 0.20 because predictions may be distributed unevenly across classes.

4 Models and evaluation protocol

The CNN follows the sentence-classification pattern of convolution and max pooling [4]. The RNN is a bidirectional LSTM [5]. Both learn randomly initialized 128-dimensional embeddings and optimize five-class cross-entropy. No pretrained transformer or external embeddings are used.

Table 5. Implemented architectures and common training settings.

Setting	CNN	BiLSTM
Text encoder	1D windows 3, 4, 5; 128 filters each	1 bidirectional layer; 128 hidden units per direction
Representation	ReLU, masked max pooling, 384-d concatenation	Concatenated final forward and backward states, 256-d
Output	Dropout 0.3; linear five-class layer	Dropout 0.3; linear five-class layer
Main parameter count	4,038,917	4,105,477
Optimization	Adam, learning rate 0.001; batch 64	Same
Stopping	Up to 15 epochs; patience 3 on validation macro-F1	Same
Other controls	Gradient norm clip 1.0; deterministic seed settings	Same

Adam [6] updates all trainable parameters. The checkpoint with the highest validation macro-F1 is selected, then evaluated on the test set. The configured architecture and training settings are held fixed for the planned comparisons. A maximum-epoch stop is not proof of convergence: the BiLSTM input-plus-output run with seed 42 selects epoch 15, at the training cap.

There are 23 distinct completed results: 14 main neural runs, four cross-domain CNN runs, four additional CNN ablations, and one interpretable-feature logistic regression. Output-only and input-plus-output main conditions use seeds 42, 43, and 44. The two input-only controls, all cross-domain experiments, and all ablations use seed 42. The main CNN output-only seed-42 fit is reused as the full-test ablation reference rather than counted as another run.

4 1 Metrics and uncertainty

Accuracy is the proportion of correctly attributed responses. For each source, precision and recall determine its F1; macro-F1 is the unweighted mean of the five source-specific F1 scores, with zero used for undefined cases. Class balance makes accuracy easy to interpret, but macro-F1 also exposes uneven class failures. Per-class and per-domain metrics accompany the overall scores.

For the four main conditions containing response text, results report the mean and sample standard deviation across three training seeds. Confidence intervals use 500 percentile bootstrap resamples of whole test group IDs, retaining all five paired responses. Paired deltas use identical test records in each arm and resample the same groups. These intervals condition on fitted models; they do not include training-seed uncertainty. The several intervals are not adjusted for multiple comparisons. Additional one-seed experiments are sensitivity analyses, not evidence of universal effects.

An independent review reconstructed all 23 accuracy and macro-F1 values from saved per-example predictions, checked record alignment and data hashes, and inspected the run configurations and manifests. This is an internal-consistency audit of saved evidence, without reloading weights or independently rerunning training or inference. Complete per-run scores and stopping epochs appear in Appendix A.

5 RQ1 and main classification results

Table 6. Test results on the same 8,635 responses. ± denotes across-seed SD, not a confidence interval.

Model	Input condition	Seeds	Accuracy %	Macro F1
CNN	Input only	1	20.00	0.1832
CNN	Output only	3	86.71 ± 0.51	0.8667 ± 0.0049
CNN	Input plus output	3	86.50 ± 0.16	0.8648 ± 0.0013
BiLSTM	Input only	1	20.00	0.1524
BiLSTM	Output only	3	85.67 ± 0.43	0.8571 ± 0.0039
BiLSTM	Input plus output	3	84.77 ± 0.54	0.8477 ± 0.0048

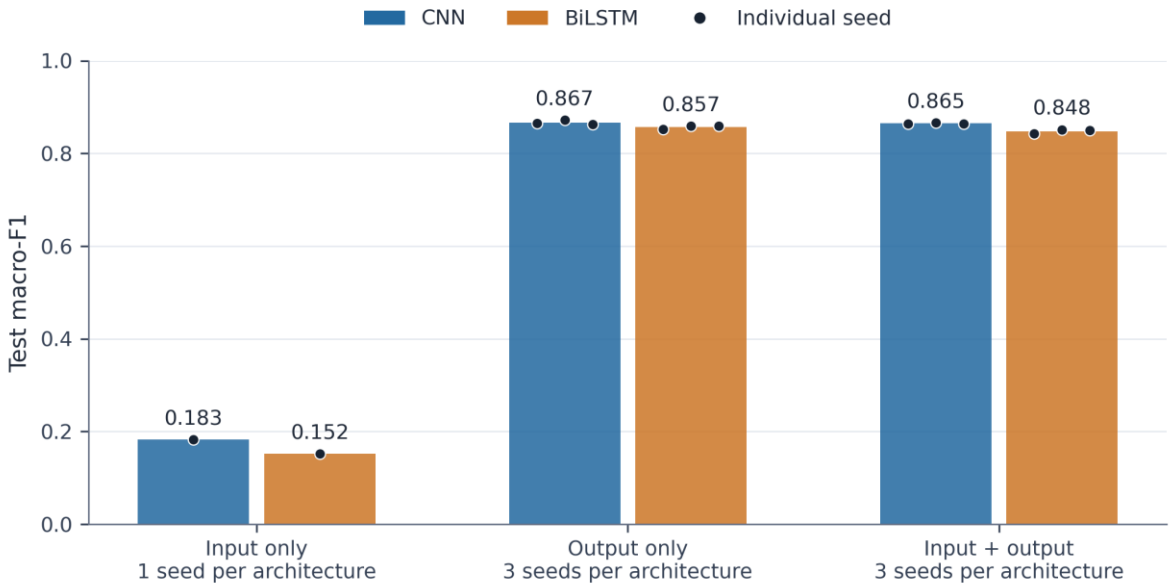


Figure 1. Mean macro-F1 and individual seed results. Input-only has one seed; each response-bearing condition has three. Macro-F1 is not constrained to the 20% input-only accuracy.

Output text alone provides a strong source signal: CNN accuracy is 86.71% and BiLSTM accuracy is 85.67%, compared with the balanced-class accuracy baseline of 20%. CNN output-only macro-F1 is higher by 0.0097 on average, with the same direction in all three seeds. This is a modest advantage under the implemented architectures and training budget, not a general claim that CNNs outperform recurrent models.

Both input-only controls attain exactly 20% accuracy, consistent with the paired design. Their lower and different macro-F1 values reflect prediction imbalance rather than a violation of the control. Neither the input-only scores nor the attribution scores evaluate whether the original LLM answers are factually correct.

5 1 Errors differ by source and task

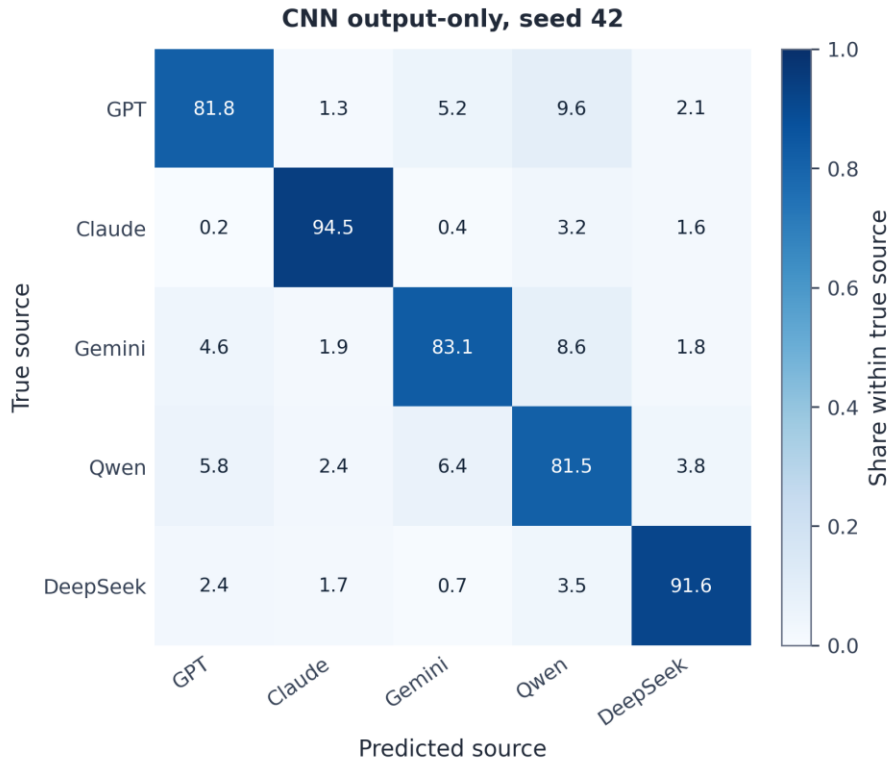


Figure 2. Row-normalized CNN output-only confusion matrix, seed 42. Each source has 1,727 test responses. Diagonal values are recalls. Seed 42 is shown for a fixed illustrative fit, not selected as the best seed.

Across three CNN seeds, class F1 is highest for Claude (0.9330) and DeepSeek (0.9133), followed by Gemini (0.8506), GPT (0.8425), and Qwen (0.7942). In Figure 2, 166 GPT and 148 Gemini responses are assigned to Qwen. These asymmetric confusions indicate overlapping response patterns; they do not establish a particular causal style feature or a source-model quality ranking.

Table 7. Output-only domain macro-F1, mean ± SD over three seeds.

Domain	Test rows	CNN	BiLSTM
Knowledge and reasoning	3,920	0.9673 ± 0.0013	0.9491 ± 0.0044
Factual QA	3,235	0.7490 ± 0.0108	0.7454 ± 0.0025
Coding	990	0.8883 ± 0.0041	0.8935 ± 0.0070
Mathematics	300	0.9021 ± 0.0139	0.8791 ± 0.0117
Creative writing	190	0.6099 ± 0.0196	0.6430 ± 0.0216

The aggregate result is dominated by knowledge/reasoning and factual QA; it is not an equally weighted task average. CNN is strongest overall but BiLSTM is better on coding and creative writing. Creative writing is the most difficult domain and contains only 38 test prompts, so its estimates are especially uncertain. The gap between knowledge/reasoning and factual QA motivates the transfer experiment rather than treating a single overall number as sufficient.

6 RQ2 Effect of adding the prompt

Adding the prompt preserves the same first 512 response tokens. Therefore, any measured difference is not caused by allocating less response text to make room for the prompt. The paired comparison is input-plus-output minus output-only, using the same architecture, seed, and test records.

Table 8. Paired changes in macro-F1. Intervals are conditional 95% group-bootstrap intervals.

Model	Seed	Change	95% interval
CNN	42	-0.0014	[-0.0070, +0.0039]
CNN	43	-0.0060	[-0.0121, -0.0003]
CNN	44	+0.0017	[-0.0039, +0.0077]
BiLSTM	42	-0.0103	[-0.0185, -0.0033]
BiLSTM	43	-0.0077	[-0.0141, -0.0015]
BiLSTM	44	-0.0101	[-0.0165, -0.0033]

For CNN, mean macro-F1 changes from 0.8667 to 0.8648, a difference of -0.0019. The sign varies across seeds, and two of the three intervals include zero. The appropriate conclusion is that this experiment detects no robust benefit from concatenating the prompt. It does not prove that a different architecture or context representation could never benefit.

For BiLSTM, macro-F1 falls from 0.8571 to 0.8477, a mean change of -0.0093. All three fitted-seed intervals are below zero. The longer combined sequence increases the burden on the final recurrent representation and on optimization, which are plausible explanations rather than tested mechanisms. The seed-42 combined-input run reaches the 15-epoch cap, so the conclusion is conditional on this fixed training budget.

6 1 Computational cost

Table 9. Mean recorded runtime per main response-bearing run in the recorded CPU environment.

Architecture	Output only	Input plus output
CNN	22.23 minutes	35.56 minutes
BiLSTM	458.15 minutes	1,349.44 minutes

The recorded environment is Windows 11, Python 3.12.10, PyTorch 2.14.1+cpu, four CPU threads, and no available CUDA device. The 22 neural runs total approximately 96.68 recorded hours. The timing field includes training and the initial test pass, but excludes later bootstrap/report generation; it is not a complete end-to-end stopwatch or a controlled hardware benchmark.

The two main architectures have similar parameter counts, yet recurrent processing is substantially slower in this implementation. Combined with its slightly stronger overall accuracy, the output-only CNN offers the better observed accuracy/runtime tradeoff. This practical conclusion is specific to the tested code, machine, and stopping behavior.

Saved histories also justify validation-based selection. For CNN output-only seed 42, training cross-entropy falls from 0.2225 at selected epoch 7 to 0.1510 at epoch 10, while validation loss rises from 0.3722 to 0.4249. The reported test result uses epoch 7, preserving the validation-selected checkpoint despite continued training-loss improvement.

7 RQ3 Transfer to unseen task domains

For each target domain, two output-only CNNs are trained with seed 42. The held-out condition removes every connected training and validation group that contains the target domain. The matched-mixed condition may include target-domain training examples but is downsampled to the same number of training prompts. Both conditions use the same non-target validation set and identical target-domain test set. Their vocabularies are fitted separately from permitted training text.

Table 10. Cross-domain design counts. Each prompt contributes five responses.

Target	Train prompts per arm	Target prompts in mixed train	Validation prompts	Test prompts
Coding	7,140	797	1,540	198
Factual QA	5,041	1,887	1,080	647

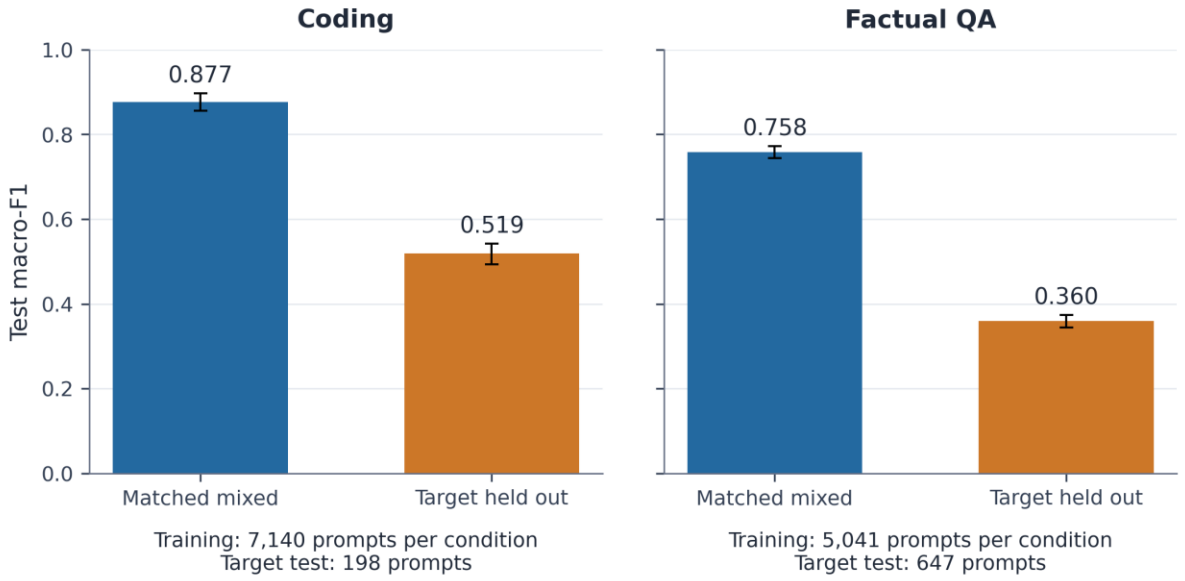


Figure 3. Matched-mixed versus target-held-out macro-F1. Error bars are conditional 95% group-bootstrap intervals; paired-difference intervals are reported below.

Coding macro-F1 falls from 0.8769 to 0.5187: change -0.3582, 95% interval [-0.3955, -0.3245]. Factual-QA macro-F1 falls from 0.7580 to 0.3599: change -0.3981, interval [-0.4179, -0.3792]. Accuracy falls from 87.58% to 57.68% for coding and from 75.64% to 37.71% for factual QA. The decline cannot be attributed simply to a smaller number of training prompts because that count is matched.

Errors are uneven across sources. On held-out coding, Qwen recall is 12.12% and GPT recall is 16.16%; 151 of 198 Qwen answers are predicted as Claude. On held-out factual QA, Qwen recall is 11.75% and DeepSeek recall is 17.77%. Some source information transfers, but the large reductions show that within-distribution accuracy does not imply a domain-invariant generator signature.

This experiment covers two broad target domains, one CNN architecture, and one seed. Its intervals do not measure variability from training-sample selection or other seeds. It changes task and benchmark exposure together; it cannot separate every aspect of domain shift from collection-specific response patterns.

8 RQ4 Formatting keywords and interpretable features

The full-test ablations retrain output-only CNNs with seed 42 after transforming the original first-512-token span. Deleted tokens are not replaced with later content. These comparisons measure learnability under each representation, rather than edited-input sensitivity of one fixed classifier.

Table 11. Full-test ablations on 8,635 identical test records. Reference macro-F1 is 0.8652.

Condition	Macro F1	Change from raw	95% interval
Original prefix	0.8652	Reference	-
Formatting removed	0.8283	-0.0369	[-0.0434, -0.0297]
Model keywords masked	0.8686	+0.0034	[-0.0019, +0.0087]

Removing approximate Markdown structures, code-fence delimiters, list/heading markers, and whitespace layout lowers macro-F1 by 0.0369 (3.69 points on a 0-to-100 scale). Structure is informative, while substantial signal remains. The rules also change token counts and may remove syntax or content, preventing a pure causal interpretation of formatting.

Masking replaces the five model names and covered provider variants with one identity marker, independently of the true label. No detectable loss appears; indirect identity clues may remain. Matches occur in only 168 of 40,295 training prefixes (0.417%), limiting expected aggregate impact, and need not be self-identifications.

8 1 A transparent feature baseline

Multinomial logistic regression uses 13 response features: word and character counts, mean word length, approximate sentence length, paragraph count, heading/list rates, code-fence count, question/exclamation rates, digit and punctuation rates, and lexical type-token ratio (TTR). All predictive features use the same original response prefix available to the neural baseline. Median imputation, a missing-value indicator, and scaling are fitted on training data only. C is fixed at 1.0 rather than tuned on the test set.

The feature classifier achieves 52.58% accuracy and 0.5247 macro-F1 (validation macro-F1 0.5141), showing that simple observable properties carry source information. Neural models outperform this linear feature baseline, but the gain could reflect richer text information, nonlinear combinations of cues, or both. Correlated feature coefficients are associations, not independent importance scores or explanations of the CNN’s internal decisions.

Table 12. Training-only descriptions. Full-response lengths and paired first-128-word TTR have different scopes.

Source	Mean words	Median words	Paired TTR
GPT	205.3	119	0.658
Claude	237.7	201	0.603
Gemini	1494.6	229	0.577
Qwen	674.4	208	0.613
DeepSeek	359.2	217	0.599

Length is strongly right-skewed, especially for Gemini and Qwen. TTR uses the first 128 whitespace words on 3,622 training prompts for which all five sources are eligible; tokens that become empty after normalization are dropped. The resulting mean denominators vary from 108.73 for GPT to about 115-119 for the others; higher TTR therefore does not establish richer vocabulary. Full-response lengths are descriptive, not the prefix-limited classifier inputs.

8 2 Controlling observed sequence length

The fixed-length experiment chooses the largest K in {32, 64, 128, 256} that retains at least 30% and at least 100 training prompts, using training lengths only. K = 256 is selected. A connected group is eligible only when every response has at least K tokenizer tokens. The same rule is then applied to validation and test. Two fresh CNNs compare the original prefix of up to 512 tokens with exactly the first 256 tokens on identical retained records.

Table 13. Retention under the fixed-length eligibility rule.

Split	Retained prompts	Retained responses	Prompt retention
Train	3,281	16,405	40.71%
Validation	660	3,300	38.22%
Test	704	3,520	40.76%

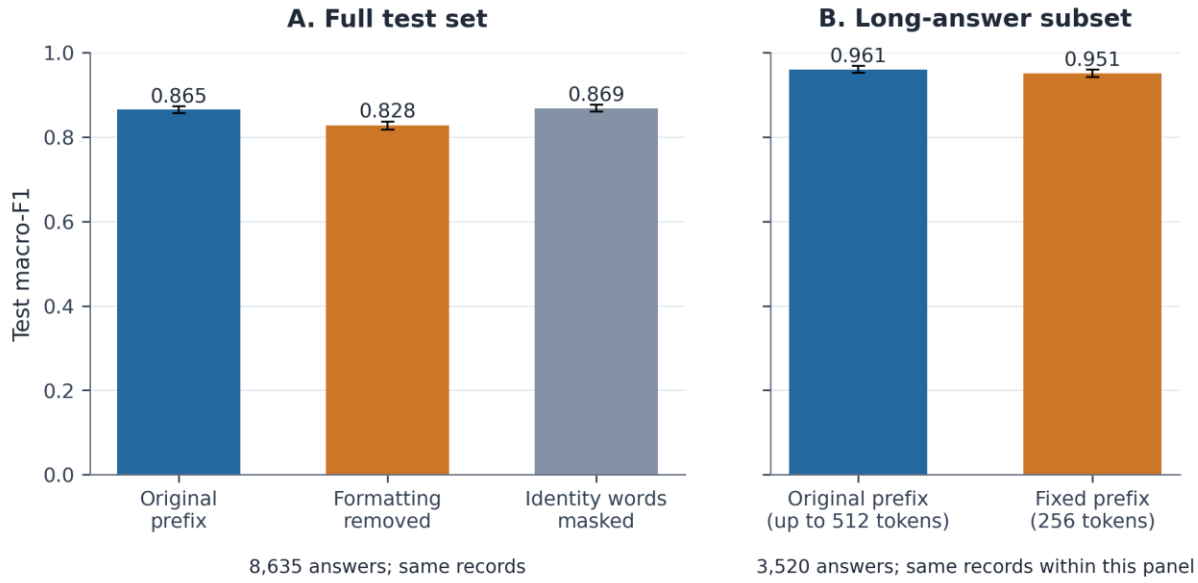


Figure 4. The two panels represent different training and test populations. Only within-panel contrasts are controlled comparisons. Error bars are conditional 95% group-bootstrap intervals.

On the long-answer subset, macro-F1 changes from 0.9605 for the original prefix to 0.9514 for exactly 256 tokens. The paired change is -0.0091, with interval [-0.0160, -0.0024]. Strong classification remains when observed tokenizer sequence length is constant, supporting source information beyond variation in that length within this selected population.

Selection materially changes the task mixture: 2,560 of 3,520 retained test responses are knowledge/reasoning (72.73%), while only five are factual QA (one prompt), compared with 3,235 factual-QA responses in the full test set. Consequently, the approximately 95% subset score cannot be interpreted as an improvement over the approximately 87% full-test score. Nor is the five-row factual-QA subset large enough to support a domain-specific conclusion.

Truncation removes content as well as length variation. Equal tokenizer-token counts do not equalize word counts, characters, formatting, semantics, or full original response length. This experiment is a conditional sensitivity check, not a causal estimate of how much of the full-data performance is explained by verbosity.

9 Concrete errors and limits of interpretation

Post hoc error inspection joins saved predictions to the verified test CSV by record_id. Examples below illustrate failure modes after evaluation; they are not used to filter the test set or choose a new model. The accompanying case file preserves full identifiers for traceability.

9 1 Identical answers can make attribution ambiguous

For a SimpleQA question about the founding year of Cáceres in Antioquia, Gemini and Qwen produce exactly the same response: “The municipality of Cáceres, Antioquia, Colombia, was founded in **1576**.” The seed-42 CNN predicts Gemini for both, with the same maximum softmax value of 0.7852. Thus Qwen’s answer is misclassified even though no text-only deterministic classifier can distinguish these two identical strings. Shared short factual answers are a real limitation of attribution, not necessarily a training defect.

In another SimpleQA case concerning the cemetery where Camille Pissarro’s youngest son was buried, a Qwen response is classified as GPT with maximum softmax 0.9949. A large softmax value does not ensure a correct source attribution. These probabilities were not calibrated, and the project does not validate a confidence threshold for real-world decisions.

9 2 Scope and threats to validity

The five labels represent one specified model version/configuration each. Provider settings, prompt templates, refusal behavior, answer formatting, and collection choices may all influence classification. Visible prompt equality does not establish identical hidden system instructions or decoding settings. The model can learn collection-specific regularities rather than an immutable family fingerprint.

Prompt-level splitting and lexical near-duplicate grouping address known leakage routes, but do not guarantee complete semantic independence. The benchmark mixture is uneven, and a response-level aggregate can hide weak performance on smaller or shifted domains. No later collection period, unseen generator, human-text class, or translation condition is tested. There is no robustness evaluation of a fixed classifier on externally edited or paraphrased outputs.

Three seeds characterize only the main response-bearing comparisons. Cross-domain and ablation experiments use one seed and conditional test-group intervals; these do not capture all retraining variability. Approximate linguistic features behave differently for prose, mathematics, and code. Results from short or highly selected subsets require especially narrow interpretation.

10 Lessons learned and next steps

The central methodological lesson is that a useful classifier score depends on the evaluation design. Pairing the same question across generators makes class balance meaningful and creates an exact negative control. Grouped splits reduce question leakage. Holding the output budget and test records fixed makes the input and ablation comparisons interpretable. The large cross-domain losses show why random held-out test performance alone is insufficient.

The practical lesson is to plan computation before scaling experiments. Similar parameter counts do not imply similar runtime: recurrent sequence processing dominates the recorded CPU budget. A short smoke run, epoch timing, validation-based stopping, saved histories, and restartable checkpoints make long runs manageable. The resulting logs support this report without another training cycle.

A follow-up study should first replicate domain and ablation effects across more seeds, then evaluate a fresh collection with matched generation settings and additional versions per family. Longer-context alternatives, probability calibration, and unknown-source rejection would address limitations of the current closed-set classifier. Those experiments are future work, not completed results.

11 Reproducibility and conclusion

The repository package contains the Python data-preparation and training code, fixed configuration, tests, environment records, per-run metrics and predictions, aggregate tables, selected figures, and an independent verification script. The data archive is externally downloadable; processed text and trained checkpoint files are not bundled in the compact repository package. The full prepared data can be recreated from the pinned source revision and checked against the saved hashes. DATA.md and README.md give the exact commands and output locations.

The frozen dataset revision is 0e5af1b84bf73437a01a1849c0f1d2468baa93fc. The source archive SHA256 is b79f8cde1a6f029c2efa663a3a3b6f7748defb22341fe59f328cebef6648c8f1. Run configurations retain architecture, seed, text budgets, transformations, and checkpoint-selection information. Saved predictions permit recalculation of every reported accuracy and macro-F1 without rerunning model training. Fresh inference requires a retained local checkpoint or a reproducible retraining run.

The results answer all four research questions: output-only attribution is strong within this benchmark mixture; prompts alone carry no label information under the paired design and concatenating them does not improve the tested models overall; cross-domain transfer is substantially weaker; and observable response features support above-chance attribution, with sensitivity to formatting normalization. The most defensible conclusion is successful attribution within a defined data collection, with limited evidence for general-purpose or domain-invariant model identification.

11 1 Assistance and author verification

ChatGPT/Codex assisted with study design, Python implementation and debugging, result checks, interpretation, and drafting this report. The formal experiments were run by the student in a local Windows environment. This disclosure describes the assistance used; it does not replace the student's responsibility to verify the analysis, understand the code, and explain the submitted work under the course policy [1].

References

[1] CMPSC 448. Midterm Project: Who Wrote It? Identifying LLMs from Their Responses. Course project specification, modified September 17, 2026. Instructor-provided PDF.

[2] H. Li et al. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing. arXiv:2601.07206, 2026. https://arxiv.org/abs/2601.07206

[3] LLMRouterBench authors. Code and response dataset. https://github.com/ynulihao/LLMRouterBench ; https://huggingface.co/datasets/NPULH/LLMRouterBench . Frozen data revision stated above. Accessed October 10, 2026.

[4] Y. Kim. Convolutional Neural Networks for Sentence Classification. Proceedings of EMNLP, pp. 1746-1751, 2014. https://doi.org/10.3115/v1/D14-1181

[5] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8), pp. 1735-1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735

[6] D. P. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. ICLR, 2015. https://arxiv.org/abs/1412.6980

Appendix A Complete experiment inventory

Every row is a distinct completed result. Accuracy is shown as a percentage; macro-F1 is on a 0-to-1 scale. Best/total gives the selected validation epoch and completed training epochs. Minutes use each run's recorded timing field. Single-run results are not averaged across seeds.

Experiment	Seed	Test n	Acc %	F1	Best/total	Minutes
CNN input	42	8,635	20.00	0.1832	5/8	9.9
CNN output	42	8,635	86.51	0.8652	7/10	23.8
CNN output	43	8,635	87.28	0.8722	6/9	20.2
CNN output	44	8,635	86.33	0.8627	7/10	22.6
CNN both	42	8,635	86.37	0.8638	9/12	40.9
CNN both	43	8,635	86.67	0.8663	9/12	39.4
CNN both	44	8,635	86.45	0.8644	5/8	26.4
BiLSTM input	42	8,635	20.00	0.1524	3/6	62.5
BiLSTM output	42	8,635	85.18	0.8526	9/12	434.5
BiLSTM output	43	8,635	85.94	0.8591	9/12	439.0
BiLSTM output	44	8,635	85.88	0.8594	10/13	500.9
BiLSTM both	42	8,635	84.17	0.8423	15/15	1430.9
BiLSTM both	43	8,635	85.21	0.8515	10/13	1244.6
BiLSTM both	44	8,635	84.92	0.8494	11/14	1372.8
Coding mixed	42	990	87.58	0.8769	6/9	18.3
Coding held out	42	990	57.68	0.5187	6/9	18.9
Factual QA mixed	42	3,235	75.64	0.7580	5/8	11.7
Factual QA held out	42	3,235	37.71	0.3599	9/12	17.1
CNN no format	42	8,635	82.74	0.8283	5/8	18.5
CNN keyword mask	42	8,635	86.81	0.8686	9/12	29.0
CNN long subset raw	42	3,520	96.05	0.9605	8/11	11.3
CNN fixed 256	42	3,520	95.14	0.9514	12/15	7.2
Feature logistic	42	8,635	52.58	0.5247	-	-

Main runs use 40,295 training responses and 8,635 validation responses. Cross-domain training and validation sizes follow Table 10 (multiply prompts by five). Both length-subset runs use 16,405 training and 3,300 validation responses. Other full-test ablations and the feature baseline use the original splits. The original CNN output seed-42 row also serves as the reference for no-format and keyword masking; it is not an additional fit.

Evidence map: tables/main_results.csv and all_run_metrics.csv support the main and inventory tables; rq2_paired_deltas.csv, rq3_paired_deltas.csv, and rq4_paired_deltas.csv support paired intervals; rq3_design and rq4_design specify matched populations; per-run predictions and confusion matrices support error analysis; descriptive/tables contains training-only linguistic summaries. The independent audit and case-provenance files accompany the package.